



CROWDSHIELD - COMPLETE SYSTEM SPECIFICATION FOR DEVELOPMENT



EXECUTIVE SUMMARY

CrowdShield is a real-time cyber threat intelligence web application that uses AI-powered analysis, community validation, and predictive analytics to detect, track, and prevent online scams. It operates on a **freemium model** with public features available to everyone and advanced analyst tools behind a subscription paywall.



SYSTEM ARCHITECTURE

Technology Stack

Frontend:

- React 18.3.1 (UI Framework)
- TypeScript (Type Safety)
- Tailwind CSS v4 (Styling)
- Lucide React (Icons)
- Recharts (Data Visualization)
- Motion/Framer Motion (Animations)
- React Router (Navigation)

Backend:

- Convex (Database & Backend)
- Azure AI Foundry (AI Model Hosting)
- Stripe (Payment Processing)
- SendGrid/AWS SES (Email Service)
- Cloudflare (CDN & Security)

Infrastructure:

- Vercel/Azure App Service (Hosting)
- AWS S3 (File Storage)
- Cloudflare (WAF/DDoS Protection)
- DataDog (Monitoring)



USER TYPES & PERMISSIONS

1. Public Users (Free - No Login Required)

Access Level: Basic

What They Can Do:

- Submit scam reports

- Get AI analysis of suspicious content
- View live threat feed (all community reports)
- Vote on reports (Scam/Suspicious/Safe)
- View location-based threats
- Subscribe to newsletter
- Track personal contribution stats

What They Cannot Do:

- Access geo intelligence maps
- View campaign tracking
- See advanced analytics
- Launch simulations
- Export data
- Receive priority alerts

2. Analysts (Paid Subscription)

Access Level: Professional

Subscription Options:

- Monthly: \$49/month
- Annual: \$29/month (\$348/year - 40% savings)

All Public Features PLUS:

Advanced Analytics:

- Full dashboard with 4 key metrics
- Interactive geo intelligence maps with heatmaps
- Campaign tracking & propagation analysis
- Advanced charts (line, pie, bar, heatmap grid)
- Nowcast predictions (6-24 hour forecasts)
- Regional intelligence reports
- Export capabilities

Professional Tools:

- Launch attack simulations for training

- Priority threat alerts
- Alert center with filtering
- Set regional alerts
- Track campaign velocity
- View propagation timelines

Data Access:

- All reports (not just verified)
- Historical data analysis
- Trend analysis
- Velocity calculations
- Pattern recognition results

3. Administrators (Internal Team)

Access Level: Full System Control

All Analyst Features PLUS:

User Management:

- View all users with detailed profiles
- See subscription status
- Monitor user activity
- Suspend/activate accounts
- View user statistics
- Track conversion rates

Report Moderation:

- Review all submitted reports
- Verify or reject reports
- Mark false positives
- Remove inappropriate content
- Bulk actions
- View AI analysis details

Subscription Management:

- View all subscriptions
- See MRR (Monthly Recurring Revenue)
- Track churn rate
- Cancel subscriptions
- Retry failed payments
- View payment methods
- Export subscription data

Customer Support:

- View support messages/tickets
- Reply to user inquiries
- Mark tickets as resolved
- Priority management
- Conversation threading
- Track response times

Communications:

- Send announcements to all users
- Target specific user groups (All/Analysts/Free)
- Compose newsletters
- Manage newsletter subscribers
- View email analytics (open rate, click rate)
- Newsletter history

System Monitoring:

- Dashboard with system health
- API status
- Database performance
- AI processing load
- Storage usage
- Server uptime
- Critical alerts



COMPLETE USER FLOWS

Flow 1: Public User Reports a Scam

1. User visits website (no login needed)
2. Clicks "Report a Scam" or navigates to Submit page
3. Fills out form:
 - Pastes suspicious message/URL
 - Optionally uploads screenshot
 - Selects platform (WhatsApp/SMS/Email/etc.)
 - Optionally adds region
 - Optionally adds context
4. Clicks "Analyze with AI"
5. System:
 - Shows loading animation (2 seconds)
 - Sends content to Azure AI for analysis
 - AI returns:
 - * Threat score (0-100)
 - * Severity level (Low/Medium/High/Critical)
 - * Scam type classification
 - * Reasoning explanation
 - * Detected attack patterns
 - * Safety recommendations
6. User sees result card with:
 - AI score ring (colored by severity)
 - Severity badge with glow effect
 - Scam type identification
 - Detailed reasoning
 - Attack patterns (pills)
 - Safety recommendations (checklist)
 - Confirmation: "Added to live feed"
 - Vote buttons (optional)
7. Report saved to database
8. Report appears in community feed
9. User can vote on their own and others' reports

Flow 2: User Subscribes to Newsletter

1. User scrolls to bottom of landing page
2. Sees newsletter signup form
3. Enters email address
4. Clicks "Subscribe"
5. System:
 - Validates email format
 - Saves to database
 - Sends confirmation email
 - Shows success message
6. User receives:
 - Immediate confirmation on screen
 - Welcome email
 - Future weekly newsletters from admin

Flow 3: User Views Location-Based Threats

1. User clicks "Near Me" in navigation
2. System requests browser geolocation permission
3. Two scenarios:

A. Permission Granted:

- Browser provides latitude/longitude
- System reverse geocodes to city name
- Filters reports within 50km radius
- Calculates local threat level
- Identifies common scam types
- Displays:
 - * Location name
 - * Threat level badge
 - * Active threats count
 - * 24-hour increase
 - * Trend direction
 - * Common scam types (top 3)
 - * Recent local reports
 - * Safety tips for area

B. Permission Denied:

- Shows permission request screen
- Explains why location is needed
- "Enable Location" button
- Privacy reassurance message

Flow 4: User Upgrades to Analyst

1. User clicks "Upgrade to Analyst"
2. Sees pricing page with:
 - Feature comparison (Free vs Analyst)
 - Two pricing options (Monthly/Annual)
 - Demo buttons
3. Three options:

A. Try Demo (No Payment):

- Clicks "View Analyst Dashboard"
- Instant access to full analyst UI
- Yellow banner: "Demo Mode"
- Can explore all 7 screens
- "Exit Demo" returns to pricing

B. Try Admin Demo (No Payment):

- Clicks "View Admin Dashboard"
- Instant access to admin panel
- Purple "ADMIN ACCESS" badge
- Can explore all 8 admin screens
- "Exit Demo" returns to pricing

C. Purchase Subscription:

- Clicks "Start Monthly Plan" or "Start Annual Plan"

- Payment modal opens
- Enters:
 - * Card number
 - * Expiry date (MM/YY)
 - * CVC
 - * Cardholder name
- Clicks "Pay \$XX"
- System:
 - * Validates card (Stripe)
 - * Processes payment
 - * Creates subscription record
 - * Updates user role to "analyst"
 - * Shows success message
- User redirected to analyst dashboard
- Full access granted immediately

Flow 5: Analyst Uses Geo Intelligence Map

1. Analyst logs in
2. Navigates to "Geo Map" from sidebar
3. Sees full-screen interactive map:
 - Dark vector map background
 - Heatmap layer (green → amber → red)
 - Cluster circles with report counts
 - Pulsing animations on critical zones
4. Analyst can:
 - Toggle layers (Heatmap/Clusters/Campaigns)
 - Filter by time (1h/24h/7d)
 - Search for specific regions
 - Click on clusters to see details
5. When region clicked:
 - Left panel slides in
 - Shows:
 - * Region name
 - * Total reports
 - * Top scam type
 - * Severity level
 - * Trend (rising/stable/falling)
 - * Recent reports in area
 - "Set alert for this region" button
6. Legend shows color scale
7. Can launch simulation from this view

Flow 6: Admin Manages Subscriptions

1. Admin logs into admin panel
2. Navigates to "Subscriptions"
3. Sees dashboard:
 - Active subscriptions: 847
 - MRR: \$38,403
 - Past due: 4

- Churn rate: 12.5%
- 4. Views subscription table:
 - All customer subscriptions
 - Status, plan, amount, next billing
- 5. Admin actions:

- A. Cancel Subscription:
 - Clicks cancel button
 - Confirmation dialog
 - System:
 - * Updates status to "cancelled"
 - * Schedules end date
 - * Sends email to user
 - * Pro-rates refund if annual

- B. Retry Failed Payment:
 - For "Past Due" subscriptions
 - Clicks retry button
 - System:
 - * Attempts to charge card again
 - * Updates status on success/failure
 - * Sends email notification

- C. Export Data:
 - Clicks "Export Data"
 - Downloads CSV with all subscriptions
 - Includes full payment history

Flow 7: Admin Replies to Support Message

1. Admin navigates to "Messages"
2. Sees inbox with message list:
 - New messages highlighted
 - Priority indicators (High/Medium/Low)
 - Status badges (New/Replied/Closed)
3. Clicks on a message
4. Right panel shows:
 - Full message thread
 - User information
 - Timestamp
 - Priority and status
5. Admin reads message
6. Types reply in text area
7. Clicks "Send Reply"
8. System:
 - Saves reply to database
 - Updates message status to "Replied"
 - Sends email notification to user
 - Shows confirmation
9. User receives:
 - Email with admin reply

- Can respond via email
 - Response appears in same thread
10. Admin can mark as closed when resolved

Flow 8: Admin Sends Announcement

1. Admin navigates to "Announcements"
2. Fills out form:
 - Title: "Critical Security Update"
 - Message: "New phishing campaign detected..."
 - Selects recipients: "All Users" (15,293)
3. Clicks "Send to 15,293 Users"
4. System:
 - Shows loading state (2 seconds)
 - Creates announcement record
 - Queues emails to SendGrid/AWS SES
 - Sends emails in batches
 - Tracks delivery
 - Shows success confirmation
5. All users receive:
 - Email notification
 - In-app notification (if logged in)
 - Banner on dashboard (for analysts)
6. Admin can track:
 - Delivery count
 - Open rate
 - Click rate (if links included)
7. Announcement appears in "Recent Announcements" panel

Flow 9: Admin Sends Newsletter

1. Admin navigates to "Newsletter"
2. Tab: "Compose Newsletter"
3. Fills out:
 - Subject: "Weekly Threat Digest – May 2024"
 - Content: "This week we saw a 23% increase..."
4. Clicks "Send to X Subscribers"
5. System:
 - Validates content
 - Retrieves active subscribers from database
 - Sends via email service
 - Tracks opens and clicks
 - Updates statistics
6. Subscribers receive email
7. Admin can:
 - View delivery confirmation
 - Track open rate in real-time
 - See click rate
 - View in "History" tab
8. Analytics updated:
 - Total sent

- Open rate %
- Click rate %
- Top clicked links

DATABASE SCHEMA - DETAILED

Table 1: users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Authentication  
  email VARCHAR(255) UNIQUE, -- Can be null for anonymous users  
  password_hash VARCHAR(255), -- Hashed with bcrypt  
  username VARCHAR(100),  
  
  -- Role & Subscription  
  role ENUM('public', 'analyst', 'admin') DEFAULT 'public',  
  subscription_plan ENUM('free', 'monthly', 'annual') DEFAULT  
'free',  
  subscription_status ENUM('active', 'cancelled', 'expired',  
'past_due'),  
  subscription_start TIMESTAMP,  
  subscription_end TIMESTAMP,  
  stripe_customer_id VARCHAR(255),  
  stripe_subscription_id VARCHAR(255),  
  
  -- Location  
  location_lat DECIMAL(10, 8),  
  location_lng DECIMAL(11, 8),  
  location_city VARCHAR(100),  
  location_country VARCHAR(100),  
  location_updated_at TIMESTAMP,  
  
  -- Activity Stats  
  reports_submitted INTEGER DEFAULT 0,  
  votes_count INTEGER DEFAULT 0,  
  accuracy_score DECIMAL(5, 2) DEFAULT 0.00, -- Voting accuracy %  
  contribution_points INTEGER DEFAULT 0,  
  
  -- Newsletter  
  newsletter_subscribed BOOLEAN DEFAULT false,  
  newsletter_subscribed_at TIMESTAMP,  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  last_login TIMESTAMP,  
  is_active BOOLEAN DEFAULT true,  
  is_email_verified BOOLEAN DEFAULT false,  
  email_verification_token VARCHAR(255),
```

```
password_reset_token VARCHAR(255),  
password_reset_expires TIMESTAMP,
```

```
-- Indexes  
INDEX idx_email (email),  
INDEX idx_role (role),  
INDEX idx_subscription_status (subscription_status),  
INDEX idx_location (location_lat, location_lng)  
);
```

Purpose: Store all user accounts, authentication, subscriptions, and activity

Key Fields:

- **role**: Determines access level (public/analyst/admin)
- **subscription_plan**: Billing plan type
- **subscription_status**: Current payment status
- **accuracy_score**: Calculated from voting history
- **newsletter_subscribed**: Newsletter opt-in status

Table 2: reports

```
CREATE TABLE reports (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Submission Info  
  user_id UUID REFERENCES users(id), -- NULL for anonymous  
  ip_address VARCHAR(45), -- For anonymous tracking  
  user_agent TEXT,  
  
  -- Content  
  content TEXT NOT NULL,  
  screenshot_url VARCHAR(500),  
  screenshot_s3_key VARCHAR(500),  
  platform ENUM('whatsapp', 'sms', 'email', 'facebook',  
'instagram', 'other'),  
  additional_context TEXT,  
  
  -- Classification (from AI)  
  scam_type VARCHAR(50), -- e.g., 'Phishing', 'Smishing'  
  severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),  
  ai_score INTEGER CHECK (ai_score >= 0 AND ai_score <= 100),  
  ai_reasoning TEXT,  
  attack_patterns JSON, -- Array of detected patterns  
  
  -- Location  
  region VARCHAR(100),  
  location_lat DECIMAL(10, 8),  
  location_lng DECIMAL(11, 8),
```

```

-- Moderation
status ENUM('pending', 'verified', 'false_positive', 'removed')
DEFAULT 'pending',
moderated_by UUID REFERENCES users(id),
moderated_at TIMESTAMP,
moderation_notes TEXT,

-- Campaign Association
campaign_id UUID REFERENCES campaigns(id),

-- Community Validation
votes_scam INTEGER DEFAULT 0,
votes_suspicious INTEGER DEFAULT 0,
votes_safe INTEGER DEFAULT 0,
total_votes INTEGER DEFAULT 0,
confidence_score DECIMAL(5, 2), -- Community agreement %

-- Visibility
is_public BOOLEAN DEFAULT true,
is_featured BOOLEAN DEFAULT false,

-- Metadata
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
verified_at TIMESTAMP,

-- Indexes
INDEX idx_severity (severity),
INDEX idx_region (region),
INDEX idx_created_at (created_at DESC),
INDEX idx_campaign_id (campaign_id),
INDEX idx_status (status),
INDEX idx_location (location_lat, location_lng),
FULLTEXT INDEX idx_content (content)
);

```

Purpose: Store all scam reports submitted by users

Key Fields:

- **content**: The suspicious message/URL
- **ai_score**: 0-100 threat score from AI
- **severity**: Auto-calculated from AI score
- **attack_patterns**: JSON array of detected patterns
- **votes_***: Community validation counts
- **status**: Moderation state

Business Logic:

```
// Calculate severity from AI score
if (ai_score >= 80) severity = 'CRITICAL';
else if (ai_score >= 60) severity = 'HIGH';
else if (ai_score >= 40) severity = 'MEDIUM';
else severity = 'LOW';

// Calculate confidence score
confidence_score = (votes_scam / total_votes) * 100;
```

Table 3: votes

```
CREATE TABLE votes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Voter Info
  report_id UUID REFERENCES reports(id) NOT NULL,
  user_id UUID REFERENCES users(id), -- NULL for anonymous
  ip_address_hash VARCHAR(64), -- SHA-256 hash for anonymous

  -- Vote
  vote_type ENUM('scam', 'suspicious', 'safe') NOT NULL,

  -- Metadata
  created_at TIMESTAMP DEFAULT NOW(),
  user_agent TEXT,

  -- Constraints
  UNIQUE(report_id, user_id), -- Logged in users: one vote per report
  UNIQUE(report_id, ip_address_hash), -- Anonymous: one vote per IP

  -- Indexes
  INDEX idx_report_id (report_id),
  INDEX idx_user_id (user_id),
  INDEX idx_created_at (created_at DESC)
);
```

Purpose: Track community votes on reports

Business Logic:

```
// Prevent duplicate votes
// If user_id exists: check UNIQUE(report_id, user_id)
// If anonymous: hash IP and check UNIQUE(report_id, ip_address_hash)

// Update report totals (trigger)
ON INSERT INTO votes:
  UPDATE reports
  SET
    votes_scam = votes_scam + (vote_type = 'scam'),
```

```

        votes_suspicious = votes_suspicious + (vote_type =
'suspicious'),
        votes_safe = votes_safe + (vote_type = 'safe'),
        total_votes = total_votes + 1
WHERE id = NEW.report_id;

```

Table 4: campaigns

```

CREATE TABLE campaigns (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Campaign Info
  name VARCHAR(255) NOT NULL,
  description TEXT,
  scam_type VARCHAR(50),
  severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),

  -- Statistics
  report_count INTEGER DEFAULT 0,
  regions_affected JSON, -- Array of region names
  regions_count INTEGER DEFAULT 0,

  -- Timeline
  first_seen TIMESTAMP NOT NULL,
  last_seen TIMESTAMP NOT NULL,
  peak_time TIMESTAMP,

  -- Velocity & Trend
  spread_velocity ENUM('slow', 'medium', 'fast'),
  hourly_rate DECIMAL(8, 2), -- Reports per hour
  daily_rate DECIMAL(8, 2), -- Reports per day
  trend ENUM('rising', 'stable', 'declining'),

  -- Community Validation
  confirmed_percentage DECIMAL(5, 2), -- % community confirmed

  -- Status
  status ENUM('active', 'declining', 'resolved') DEFAULT
'active',

  -- Attribution
  created_by UUID REFERENCES users(id), -- Admin who created
  auto_detected BOOLEAN DEFAULT true, -- AI vs manual creation

  -- Metadata
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  resolved_at TIMESTAMP,

  -- Indexes
  INDEX idx_status (status),

```

```

INDEX idx_severity (severity),
INDEX idx_first_seen (first_seen DESC),
INDEX idx_scam_type (scam_type)
);

```

Purpose: Group related scam reports into campaigns

Auto-Detection Logic:

```

// Campaign detection algorithm (runs every 6 hours)
async function detectCampaigns() {
  const recentReports = await getReportsLast6Hours();

  // Group by scam_type and region
  const groups = groupBy(recentReports, ['scam_type', 'region']);

  // Identify campaigns
  for (const group of groups) {
    if (group.count >= 10 && group.timespan < 6_hours) {
      // Create or update campaign
      const campaign = {
        name: `${group.scam_type} Wave in ${group.region}`,
        scam_type: group.scam_type,
        severity: calculateMaxSeverity(group.reports),
        report_count: group.count,
        hourly_rate: group.count / 6,
        spread_velocity: calculateVelocity(group.hourly_rate),
        first_seen: group.earliest_report,
        last_seen: group.latest_report,
      };

      await saveCampaign(campaign);
    }
  }
}

function calculateVelocity(hourly_rate) {
  if (hourly_rate >= 20) return 'fast';
  if (hourly_rate >= 10) return 'medium';
  return 'slow';
}

```

Table 5: subscriptions

```

CREATE TABLE subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- User
  user_id UUID REFERENCES users(id) NOT NULL,

  -- Plan
  plan_type ENUM('monthly', 'annual') NOT NULL,

```

```
amount DECIMAL(10, 2) NOT NULL, -- $49 or $348
currency VARCHAR(3) DEFAULT 'USD',
```

```
-- Status
status ENUM('active', 'cancelled', 'expired', 'past_due',
'unpaid') NOT NULL,
cancel_at_period_end BOOLEAN DEFAULT false,
```

```
-- Billing
current_period_start TIMESTAMP NOT NULL,
current_period_end TIMESTAMP NOT NULL,
billing_cycle_anchor TIMESTAMP,
```

```
-- Payment Method
payment_method VARCHAR(50), -- 'stripe', 'paypal'
payment_method_details JSON, -- Last 4 digits, brand, etc.
```

```
-- Stripe Integration
stripe_subscription_id VARCHAR(255) UNIQUE,
stripe_customer_id VARCHAR(255),
stripe_price_id VARCHAR(255),
```

```
-- Cancellation
cancelled_at TIMESTAMP,
cancel_reason TEXT,
```

```
-- Trial
trial_start TIMESTAMP,
trial_end TIMESTAMP,
```

```
-- Metadata
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
```

```
-- Indexes
INDEX idx_user_id (user_id),
INDEX idx_status (status),
INDEX idx_stripe_subscription_id (stripe_subscription_id),
INDEX idx_current_period_end (current_period_end)
);
```

Purpose: Track all subscription records

Lifecycle States:

active → Payment successful, service active
past_due → Payment failed, retry in progress
cancelled → User cancelled, service until period end
expired → Subscription period ended
unpaid → Multiple payment failures

Table 6: payments


```

CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Relations
  subscription_id UUID REFERENCES subscriptions(id),
  user_id UUID REFERENCES users(id) NOT NULL,

  -- Payment Info
  amount DECIMAL(10, 2) NOT NULL,
  currency VARCHAR(3) DEFAULT 'USD',
  payment_method VARCHAR(50),

  -- Status
  payment_status ENUM('pending', 'completed', 'failed',
'refunded') NOT NULL,
  failure_reason TEXT,

  -- Stripe
  stripe_payment_intent_id VARCHAR(255) UNIQUE,
  stripe_charge_id VARCHAR(255),
  stripe_invoice_id VARCHAR(255),
  invoice_url VARCHAR(500),

  -- Refund
  refund_amount DECIMAL(10, 2),
  refund_reason TEXT,
  refunded_at TIMESTAMP,
  stripe_refund_id VARCHAR(255),

  -- Metadata
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  completed_at TIMESTAMP,

  -- Indexes
  INDEX idx_subscription_id (subscription_id),
  INDEX idx_user_id (user_id),
  INDEX idx_payment_status (payment_status),
  INDEX idx_created_at (created_at DESC)
);

```

Purpose: Record all payment transactions

Payment Flow:

```

// 1. User clicks "Pay $49"
// 2. Frontend calls Stripe.js
// 3. Stripe returns payment_intent_id
// 4. Backend creates payment record (status: 'pending')
// 5. Webhook from Stripe confirms payment
// 6. Update payment (status: 'completed')
// 7. Activate subscription

```

Table 7: messages

```
CREATE TABLE messages (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Ticket Info  
  user_id UUID REFERENCES users(id) NOT NULL,  
  subject VARCHAR(255) NOT NULL,  
  message TEXT NOT NULL,  
  
  -- Status  
  status ENUM('new', 'replied', 'closed') DEFAULT 'new',  
  priority ENUM('low', 'medium', 'high') DEFAULT 'medium',  
  
  -- Assignment  
  assigned_to UUID REFERENCES users(id), -- Admin assigned  
  assigned_at TIMESTAMP,  
  
  -- Replies  
  reply_count INTEGER DEFAULT 0,  
  last_reply_at TIMESTAMP,  
  last_reply_by UUID REFERENCES users(id),  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  closed_at TIMESTAMP,  
  first_response_time INTEGER, -- Seconds to first admin reply  
  
  -- Indexes  
  INDEX idx_user_id (user_id),  
  INDEX idx_status (status),  
  INDEX idx_priority (priority),  
  INDEX idx_created_at (created_at DESC)  
);
```

Table 8: message_replies

```
CREATE TABLE message_replies (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Relations  
  message_id UUID REFERENCES messages(id) NOT NULL,  
  from_user_id UUID REFERENCES users(id) NOT NULL,  
  
  -- Content  
  reply_text TEXT NOT NULL,  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT NOW(),
```

```
is_admin_reply BOOLEAN DEFAULT false,
```

```
-- Indexes
```

```
INDEX idx_message_id (message_id),
```

```
INDEX idx_created_at (created_at ASC)
```

```
);
```

Purpose: Support ticket system with threading

Table 9: announcements

```
CREATE TABLE announcements (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
-- Content
```

```
title VARCHAR(255) NOT NULL,
```

```
message TEXT NOT NULL,
```

```
-- Recipients
```

```
recipients ENUM('all', 'analysts', 'free_users') NOT NULL,
```

```
recipient_count INTEGER, -- Calculated at send time
```

```
-- Delivery Stats
```

```
sent_count INTEGER DEFAULT 0,
```

```
delivered_count INTEGER DEFAULT 0,
```

```
opened_count INTEGER DEFAULT 0,
```

```
clicked_count INTEGER DEFAULT 0,
```

```
-- Calculated Rates
```

```
delivery_rate DECIMAL(5, 2),
```

```
open_rate DECIMAL(5, 2),
```

```
click_rate DECIMAL(5, 2),
```

```
-- Sender
```

```
created_by UUID REFERENCES users(id) NOT NULL,
```

```
-- Status
```

```
status ENUM('draft', 'sending', 'sent') DEFAULT 'draft',
```

```
-- Metadata
```

```
created_at TIMESTAMP DEFAULT NOW(),
```

```
sent_at TIMESTAMP,
```

```
-- Indexes
```

```
INDEX idx_status (status),
```

```
INDEX idx_sent_at (sent_at DESC)
```

```
);
```

Purpose: Admin announcements to user groups

Table 10: newsletters

```
CREATE TABLE newsletters (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Content  
  subject VARCHAR(255) NOT NULL,  
  content TEXT NOT NULL,  
  html_content TEXT, -- Rich HTML version  
  
  -- Delivery  
  status ENUM('draft', 'scheduled', 'sending', 'sent') DEFAULT  
'draft',  
  scheduled_for TIMESTAMP,  
  sent_at TIMESTAMP,  
  
  -- Recipients (active subscribers only)  
  recipient_count INTEGER,  
  sent_count INTEGER DEFAULT 0,  
  delivered_count INTEGER DEFAULT 0,  
  opened_count INTEGER DEFAULT 0,  
  clicked_count INTEGER DEFAULT 0,  
  unsubscribed_count INTEGER DEFAULT 0,  
  
  -- Rates  
  delivery_rate DECIMAL(5, 2),  
  open_rate DECIMAL(5, 2),  
  click_rate DECIMAL(5, 2),  
  unsubscribe_rate DECIMAL(5, 2),  
  
  -- Links  
  tracked_links JSON, -- Array of {url, clicks}  
  
  -- Sender  
  created_by UUID REFERENCES users(id) NOT NULL,  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW(),  
  
  -- Indexes  
  INDEX idx_status (status),  
  INDEX idx_sent_at (sent_at DESC)  
);
```

Purpose: Newsletter campaigns

Table 11: newsletter_subscribers

```
CREATE TABLE newsletter_subscribers (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

-- Subscriber
email VARCHAR(255) UNIQUE NOT NULL,
user_id UUID REFERENCES users(id), -- NULL if not registered

-- Status
status ENUM('active', 'unsubscribed', 'bounced') DEFAULT
'active',

-- Stats
total_sent INTEGER DEFAULT 0,
total_opened INTEGER DEFAULT 0,
total_clicked INTEGER DEFAULT 0,
open_rate DECIMAL(5, 2),
click_rate DECIMAL(5, 2),

-- Unsubscribe
unsubscribed_at TIMESTAMP,
unsubscribe_reason TEXT,

-- Metadata
subscribed_at TIMESTAMP DEFAULT NOW(),
last_email_sent_at TIMESTAMP,

-- Email Service
sendgrid_contact_id VARCHAR(255),

-- Indexes
INDEX idx_email (email),
INDEX idx_status (status),
INDEX idx_subscribed_at (subscribed_at DESC)
);

```

Purpose: Newsletter subscriber list

Table 12: ai_analyses

```

CREATE TABLE ai_analyses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Report
  report_id UUID REFERENCES reports(id) UNIQUE NOT NULL,

  -- AI Model
  model_version VARCHAR(50), -- e.g., "gpt-4-turbo-2024-04"
  model_provider VARCHAR(50), -- "azure", "openai"

  -- Analysis
  threat_score INTEGER CHECK (threat_score >= 0 AND threat_score
<= 100),
  confidence_level DECIMAL(5, 2), -- AI confidence %

```

```

-- Classification
scam_type VARCHAR(50),
severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),

-- Details
reasoning TEXT,
attack_patterns JSON, -- Array of pattern names
similar_reports JSON, -- Array of similar report IDs
recommendations JSON, -- Array of safety tips

-- Features Detected
urgency_detected BOOLEAN,
link_detected BOOLEAN,
impersonation_detected BOOLEAN,
credential_request_detected BOOLEAN,

-- Performance
processing_time_ms INTEGER,
tokens_used INTEGER,
cost_usd DECIMAL(10, 4),

-- Metadata
created_at TIMESTAMP DEFAULT NOW(),

-- Indexes
INDEX idx_report_id (report_id),
INDEX idx_threat_score (threat_score DESC),
INDEX idx_created_at (created_at DESC)
);

```

Purpose: Detailed AI analysis results

AI Integration Flow:

```

async function analyzeReport(content) {
  const startTime = Date.now();

  // Call Azure OpenAI
  const response = await azureOpenAI.chat.completions.create({
    model: "gpt-4-turbo",
    messages: [
      {
        role: "system",
        content: `You are a cybersecurity expert analyzing
potential scams.
        Analyze the following message and provide:
        1. Threat score (0-100)
        2. Scam type
        3. Attack patterns detected
        4. Reasoning
        5. Safety recommendations`
      }
    ]
  });
}

```

```

    },
    {
      role: "user",
      content: content
    }
  ],
  temperature: 0.3,
  response_format: { type: "json_object" }
});

const analysis =
JSON.parse(response.choices[0].message.content);
const processingTime = Date.now() - startTime;

// Calculate severity from score
let severity;
if (analysis.threat_score >= 80) severity = 'CRITICAL';
else if (analysis.threat_score >= 60) severity = 'HIGH';
else if (analysis.threat_score >= 40) severity = 'MEDIUM';
else severity = 'LOW';

// Save to database
await db.ai_analyses.insert({
  report_id: reportId,
  model_version: "gpt-4-turbo-2024-04",
  model_provider: "azure",
  threat_score: analysis.threat_score,
  confidence_level: analysis.confidence,
  scam_type: analysis.scam_type,
  severity: severity,
  reasoning: analysis.reasoning,
  attack_patterns: analysis.patterns,
  recommendations: analysis.recommendations,
  processing_time_ms: processingTime,
  tokens_used: response.usage.total_tokens,
  cost_usd: calculateCost(response.usage)
});

return analysis;
}

```

Table 13: regions

```

CREATE TABLE regions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Location
  name VARCHAR(100) NOT NULL,
  country_code VARCHAR(3),
  country_name VARCHAR(100),
  city VARCHAR(100),

```

```

state_province VARCHAR(100),

-- Coordinates
latitude DECIMAL(10, 8),
longitude DECIMAL(11, 8),

-- Demographics
population INTEGER,
timezone VARCHAR(50),

-- Statistics (updated daily)
total_reports INTEGER DEFAULT 0,
reports_last_24h INTEGER DEFAULT 0,
reports_last_7d INTEGER DEFAULT 0,
reports_last_30d INTEGER DEFAULT 0,

-- Current State
current_severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),
top_scam_type VARCHAR(50),
trend ENUM('rising', 'stable', 'falling'),

-- Metadata
created_at TIMESTAMP DEFAULT NOW(),
updated_at TIMESTAMP DEFAULT NOW(),
last_report_at TIMESTAMP,

-- Indexes
INDEX idx_name (name),
INDEX idx_country_code (country_code),
INDEX idx_location (latitude, longitude),
INDEX idx_current_severity (current_severity)
);

```

Purpose: Geographic threat intelligence

Update Logic:

```

// Runs every hour
async function updateRegionStats() {
  const regions = await db.regions.findAll();

  for (const region of regions) {
    const reports24h = await getReportsInRadius(
      region.latitude,
      region.longitude,
      50, // km
      24 // hours
    );

    const reports7d = await getReportsInRadius(
      region.latitude,
      region.longitude,

```



```

    50,
    168 // hours (7 days)
  );

  // Calculate severity
  const criticalCount = reports24h.filter(r => r.severity ===
'CRITICAL').length;
  const highCount = reports24h.filter(r => r.severity ===
'HIGH').length;

  let severity;
  if (criticalCount >= 5) severity = 'CRITICAL';
  else if (highCount >= 10) severity = 'HIGH';
  else if (reports24h.length >= 20) severity = 'MEDIUM';
  else severity = 'LOW';

  // Determine trend
  const reports_previous_24h = await getReportsInRadius(
    region.latitude,
    region.longitude,
    50,
    48 // 24-48 hours ago
  );

  let trend;
  if (reports24h.length > reports_previous_24h.length * 1.2)
trend = 'rising';
  else if (reports24h.length < reports_previous_24h.length *
0.8) trend = 'falling';
  else trend = 'stable';

  // Top scam type
  const scamTypeCounts = countBy(reports24h, 'scam_type');
  const topScamType = Object.keys(scamTypeCounts)
    .sort((a, b) => scamTypeCounts[b] - scamTypeCounts[a])[0];

  await db.regions.update(region.id, {
    total_reports: region.total_reports + reports24h.length,
    reports_last_24h: reports24h.length,
    reports_last_7d: reports7d.length,
    current_severity: severity,
    top_scam_type: topScamType,
    trend: trend,
    updated_at: new Date()
  });
}
}

```

Table 14: nowcast_predictions

```
CREATE TABLE nowcast_predictions (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

-- Target
region_id UUID REFERENCES regions(id) NOT NULL,

-- Prediction
prediction_time TIMESTAMP NOT NULL, -- When prediction was made
forecast_horizon_hrs INTEGER NOT NULL, -- 6, 12, or 24 hours

-- Predicted Values
predicted_severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),
predicted_scam_type VARCHAR(50),
predicted_count INTEGER, -- Expected number of reports
confidence DECIMAL(5, 2), -- Model confidence %

-- Current Data (at prediction time)
current_velocity DECIMAL(8, 2), -- Reports per hour
current_count INTEGER,

-- Model
model_version VARCHAR(50),
algorithm VARCHAR(50), -- "time_series", "ml_classifier"

-- Validation (filled after forecast horizon)
actual_count INTEGER,
actual_severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'),
prediction_error DECIMAL(8, 2),
was_accurate BOOLEAN,

-- Metadata
created_at TIMESTAMP DEFAULT NOW(),
expires_at TIMESTAMP, -- prediction_time + forecast_horizon_hrs
validated_at TIMESTAMP,

-- Indexes
INDEX idx_region_id (region_id),
INDEX idx_prediction_time (prediction_time DESC),
INDEX idx_expires_at (expires_at)
);

```

Purpose: Threat forecasting

Prediction Algorithm:

```

async function generateNowcast(regionId, horizonHours) {
  const region = await db.regions.findById(regionId);

  // Get historical data
  const history = await getReportHistory(
    regionId,
    48 // last 48 hours
  );
}

```

```

// Calculate current velocity
const last6Hours = history.filter(r =>
  r.created_at > Date.now() - 6 * 3600 * 1000
);
const currentVelocity = last6Hours.length / 6; // reports/hour

// Time series prediction
const predicted = {
  count: Math.round(currentVelocity * horizonHours * 1.2), //
20% growth factor
  scamType: getMostCommonType(last6Hours),
  confidence: calculateConfidence(history)
};

// Determine predicted severity
let predictedSeverity;
if (predicted.count >= 30) predictedSeverity = 'CRITICAL';
else if (predicted.count >= 20) predictedSeverity = 'HIGH';
else if (predicted.count >= 10) predictedSeverity = 'MEDIUM';
else predictedSeverity = 'LOW';

// Save prediction
await db.nowcast_predictions.insert({
  region_id: regionId,
  prediction_time: new Date(),
  forecast_horizon_hrs: horizonHours,
  predicted_severity: predictedSeverity,
  predicted_scam_type: predicted.scamType,
  predicted_count: predicted.count,
  confidence: predicted.confidence,
  current_velocity: currentVelocity,
  current_count: last6Hours.length,
  model_version: "v2.1",
  algorithm: "time_series",
  expires_at: new Date(Date.now() + horizonHours * 3600 * 1000)
});

return predicted;
}

function calculateConfidence(history) {
  // Based on data quality and consistency
  if (history.length < 10) return 40; // Low data

  const variance = calculateVariance(history.map(h =>
h.hourly_count));
  if (variance > 50) return 60; // High variance
  if (variance > 20) return 75; // Medium variance
  return 90; // Low variance = high confidence
}

```

Table 15: activity_logs

```
CREATE TABLE activity_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- Actor  
  user_id UUID REFERENCES users(id),  
  ip_address VARCHAR(45),  
  user_agent TEXT,  
  
  -- Action  
  action_type VARCHAR(50), -- 'report_submit', 'vote', 'login',  
etc.  
  entity_type VARCHAR(50), -- 'report', 'campaign',  
'subscription'  
  entity_id UUID,  
  
  -- Details  
  metadata JSON, -- Additional context  
  
  -- Result  
  success BOOLEAN,  
  error_message TEXT,  
  
  -- Metadata  
  created_at TIMESTAMP DEFAULT NOW(),  
  
  -- Indexes  
  INDEX idx_user_id (user_id),  
  INDEX idx_action_type (action_type),  
  INDEX idx_created_at (created_at DESC)  
);
```

Purpose: Audit trail and analytics

Example Logs:

```
{  
  "action_type": "report_submit",  
  "metadata": {  
    "scam_type": "Phishing",  
    "ai_score": 94,  
    "platform": "whatsapp"  
  }  
}  
  
{  
  "action_type": "subscription_purchase",  
  "metadata": {  
    "plan": "annual",  
    "amount": 348,  
  }  
}
```

```
    "payment_method": "stripe"
  }
}
```

API ENDPOINTS

Public Endpoints (No Auth Required)

```
// Submit Report
POST /api/reports
Body: {
  content: string,
  screenshot?: File,
  platform: 'whatsapp' | 'sms' | 'email' | 'facebook' |
'instagram' | 'other',
  region?: string,
  context?: string,
  location?: { lat: number, lng: number }
}
Response: {
  reportId: string,
  aiAnalysis: {
    score: number,
    severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL',
    scamType: string,
    reasoning: string,
    patterns: string[],
    recommendations: string[]
  }
}
```

```
// Get Live Feed
GET /api/reports?limit=20&offset=0&severity=all
Response: {
  reports: Report[],
  total: number,
  hasMore: boolean
}
```

```
// Vote on Report
POST /api/reports/:id/vote
Body: { vote: 'scam' | 'suspicious' | 'safe' }
Response: { success: boolean }
```

```
// Get Location Threats
GET /api/reports/nearby?lat=6.9271&lng=79.8612&radius=50
Response: {
  threats: Report[],
  summary: {
    count: number,
    severity: string,

```

```
    topScamType: string,  
    trend: 'rising' | 'stable' | 'falling'  
  }  
}
```

```
// Subscribe to Newsletter  
POST /api/newsletter/subscribe  
Body: { email: string }  
Response: { success: boolean }
```

Analyst Endpoints (Requires Auth)

```
// Get Dashboard Stats  
GET /api/analyst/dashboard  
Response: {  
  activeThreats: number,  
  reportsLastHour: number,  
  campaignsDetected: number,  
  regionsAtRisk: number,  
  nowcast: {  
    region: string,  
    predictedSeverity: string,  
    message: string  
  }  
}
```

```
// Get Geo Map Data  
GET /api/analyst/map?timeFilter=24h  
Response: {  
  heatmap: {  
    lat: number,  
    lng: number,  
    intensity: number  
  }[],  
  clusters: {  
    lat: number,  
    lng: number,  
    count: number,  
    severity: string  
  }[],  
  regions: Region[]  
}
```

```
// Get Campaign Details  
GET /api/analyst/campaigns/:id  
Response: {  
  campaign: Campaign,  
  reports: Report[],  
  propagationGraph: {  
    nodes: { id: string, region: string, count: number }[],  
    edges: { from: string, to: string }[]  
  }  
}
```

```
    },  
    timeline: { date: string, count: number }[]  
}
```

// Get Analytics

GET /api/analyst/analytics

```
Response: {  
  volumeData: { date: string, reports: number }[],  
  scamTypeDistribution: { type: string, count: number }[],  
  regionData: { region: string, count: number }[],  
  predictions: NowcastPrediction[]  
}
```

// Export Data

POST /api/analyst/export

```
Body: { type: 'reports' | 'campaigns', format: 'csv' | 'json' }
```

```
Response: { downloadUrl: string }
```

// Launch Simulation

POST /api/analyst/simulation

```
Body: {  
  scamType: string,  
  intensity: 'low' | 'medium' | 'high',  
  duration: number,  
  regions: string[]  
}
```

```
Response: { simulationId: string }
```

Admin Endpoints (Requires Admin Role)

// Get All Users

GET /api/admin/users?role=all&search=&page=1

```
Response: {  
  users: User[],  
  total: number,  
  stats: {  
    totalUsers: number,  
    activeSubscriptions: number,  
    conversionRate: number  
  }  
}
```

// Moderate Report

PATCH /api/admin/reports/:id

```
Body: {  
  status: 'verified' | 'false_positive' | 'removed',  
  notes?: string  
}
```

```
Response: { success: boolean }
```

// Manage Subscription

POST /api/admin/subscriptions/:id/cancel

Response: { success: boolean }

POST /api/admin/subscriptions/:id/retry-payment

Response: { success: boolean, paymentStatus: string }

// Reply to Message

POST /api/admin/messages/:id/reply

Body: { replyText: string }

Response: { success: boolean, replyId: string }

PATCH /api/admin/messages/:id/close

Response: { success: boolean }

// Send Announcement

POST /api/admin/announcements

Body: {

title: string,

message: string,

recipients: 'all' | 'analysts' | 'free_users'

}

Response: {

announcementId: string,

recipientCount: number,

status: 'sending'

}

// Send Newsletter

POST /api/admin/newsletters

Body: {

subject: string,

content: string,

htmlContent?: string

}

Response: {

newsletterId: string,

subscriberCount: number,

status: 'sending'

}

// Get System Stats

GET /api/admin/system/stats

Response: {

apiHealth: 'operational' | 'degraded' | 'down',

databaseHealth: number, // %

aiProcessingLoad: number, // %

storageUsed: number, // %

uptime: number // %

}



BUSINESS LOGIC & ALGORITHMS

1. AI Threat Analysis

```
interface AIAnalysisRequest {  
  content: string;  
  platform?: string;  
  metadata?: Record<string, any>;  
}
```

```
interface AIAnalysisResponse {  
  threatScore: number; // 0-100  
  severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL';  
  scamType: string;  
  confidence: number;  
  reasoning: string;  
  attackPatterns: string[];  
  recommendations: string[];  
}
```

```
async function analyzeWithAI(request: AIAnalysisRequest):  
Promise<AIAnalysisResponse> {  
  const prompt = `  
    Analyze this message for scam indicators:
```

```
    Content: "${request.content}"  
    Platform: ${request.platform || 'unknown'}
```

```
    Provide:
```

- ```
 1. Threat score (0-100) where:
 - 0-20: Legitimate
 - 21-40: Low risk
 - 41-60: Medium risk - suspicious patterns
 - 61-80: High risk - likely scam
 - 81-100: Critical - definite scam with multiple
```

```
indicators
```

- ```
    2. Scam type classification:  
      - Phishing (fake login pages, credential theft)  
      - Smishing (SMS phishing)  
      - OTP Phishing (requesting verification codes)  
      - Job Scam (fake job offers requiring payment)  
      - Romance Scam (relationship manipulation for money)  
      - Investment Scam (too-good-to-be-true returns)  
      - Lottery Scam (fake prize winnings)  
      - Impersonation (pretending to be authority/company)
```

- ```
 3. Attack patterns detected (check for):
 - Urgency tactics ("act now", "limited time")
 - Suspicious links (shortened URLs, typosquatting)
 - Impersonation (claiming to be bank, government, etc.)
 - Credential requests (asking for passwords, OTPs)
 - Payment requests (wire transfer, gift cards)
```

- Too-good-to-be-true offers
- Poor grammar/spelling
- Emotional manipulation

4. Detailed reasoning (2-3 sentences)

5. Safety recommendations (3-5 actionable steps)

Return JSON format.

```
`;

const response = await azureOpenAI.chat.completions.create({
 model: "gpt-4-turbo",
 messages: [
 { role: "system", content: "You are a cybersecurity expert." },
 { role: "user", content: prompt }
],
 temperature: 0.3,
 response_format: { type: "json_object" }
});

const analysis =
JSON.parse(response.choices[0].message.content);

// Map score to severity
const severity =
 analysis.threatScore >= 80 ? 'CRITICAL' :
 analysis.threatScore >= 60 ? 'HIGH' :
 analysis.threatScore >= 40 ? 'MEDIUM' : 'LOW';

return {
 threatScore: analysis.threatScore,
 severity,
 scamType: analysis.scamType,
 confidence: analysis.confidence,
 reasoning: analysis.reasoning,
 attackPatterns: analysis.attackPatterns,
 recommendations: analysis.recommendations
};
}
```

## 2. Campaign Detection Algorithm

```
async function detectCampaigns() {
 const sixHoursAgo = new Date(Date.now() - 6 * 3600 * 1000);

 // Get recent reports
 const recentReports = await db.reports.find({
 created_at: { $gte: sixHoursAgo },
 status: { $ne: 'false_positive' }
 });
}
```

```

});

// Group by scam_type and region
const groups = new Map<string, Report[]>();

for (const report of recentReports) {
 const key = `${report.scam_type}:${report.region}`;
 if (!groups.has(key)) groups.set(key, []);
 groups.get(key)!.push(report);
}

// Detect campaigns
for (const [key, reports] of groups.entries()) {
 if (reports.length < 10) continue; // Threshold: 10 reports

 const [scamType, region] = key.split('::');

 // Check if campaign already exists
 const existing = await db.campaigns.findOne({
 scam_type: scamType,
 regions_affected: { $contains: region },
 status: 'active'
 });

 if (existing) {
 // Update existing campaign
 await db.campaigns.update(existing.id, {
 report_count: existing.report_count + reports.length,
 last_seen: new Date(),
 hourly_rate: reports.length / 6,
 updated_at: new Date()
 });
 } else {
 // Create new campaign
 const severity = calculateMaxSeverity(reports);
 const hourlyRate = reports.length / 6;

 await db.campaigns.insert({
 name: `${scamType} Wave in ${region}`,
 description: `Coordinated ${scamType} campaign detected`,
 scam_type: scamType,
 severity,
 report_count: reports.length,
 regions_affected: [region],
 regions_count: 1,
 first_seen: reports[0].created_at,
 last_seen: reports[reports.length - 1].created_at,
 spread_velocity: hourlyRate >= 20 ? 'fast' : hourlyRate
 });

 const severityLabel = reports.length >= 10 ? 'medium' : 'slow';
 const campaign = {
 name: `${scamType} Wave in ${region}`,
 description: `Coordinated ${scamType} campaign detected`,
 scam_type: scamType,
 severity,
 report_count: reports.length,
 regions_affected: [region],
 regions_count: 1,
 first_seen: reports[0].created_at,
 last_seen: reports[reports.length - 1].created_at,
 spread_velocity: hourlyRate >= 20 ? 'fast' : hourlyRate,
 severity_label: severityLabel,
 hourly_rate: hourlyRate,
 status: 'active',
 auto_detected: true
 };
 }
}

```

```

 });

 // Send alert to analysts
 await createAlert({
 severity,
 type: 'campaign',
 message: `New campaign detected: ${scamType} in ${region}`
 },
 {
 region
 });
 }
}
}

```

```

function calculateMaxSeverity(reports: Report[]): Severity {
 const counts = {
 CRITICAL: reports.filter(r => r.severity ===
'CRITICAL').length,
 HIGH: reports.filter(r => r.severity === 'HIGH').length,
 MEDIUM: reports.filter(r => r.severity === 'MEDIUM').length,
 LOW: reports.filter(r => r.severity === 'LOW').length
 };

```

```

 if (counts.CRITICAL > 0) return 'CRITICAL';
 if (counts.HIGH >= 5) return 'HIGH';
 if (counts.MEDIUM >= 5) return 'MEDIUM';
 return 'LOW';
}

```

### 3. Location-Based Threat Detection

```

async function getNearbyThreats(lat: number, lng: number,
radiusKm: number = 50) {
 // Haversine formula for distance calculation
 const reports = await db.raw(`
 SELECT *,
 (6371 * acos(
 cos(radians(?)) * cos(radians(location_lat)) *
 cos(radians(location_lng) - radians(?)) +
 sin(radians(?)) * sin(radians(location_lat))
)) AS distance
 FROM reports
 WHERE created_at >= NOW() - INTERVAL 24 HOUR
 HAVING distance <= ?
 ORDER BY distance ASC
 `, [lat, lng, lat, radiusKm]);

 // Calculate local threat level
 const criticalCount = reports.filter(r => r.severity ===
'CRITICAL').length;

```

```

 const highCount = reports.filter(r => r.severity ===
'HIGH').length;

 let localThreatLevel;
 if (criticalCount >= 3) localThreatLevel = 'CRITICAL';
 else if (highCount >= 5) localThreatLevel = 'HIGH';
 else if (reports.length >= 10) localThreatLevel = 'MEDIUM';
 else localThreatLevel = 'LOW';

 // Get common scam types
 const scamTypeCounts = countBy(reports, 'scam_type');
 const topScamTypes = Object.entries(scamTypeCounts)
 .sort(([,a], [,b]) => b - a)
 .slice(0, 3)
 .map([type, count] => ({ type, count }));

 // Calculate trend
 const reports_previous_24h = await
getSimilarReportsFromPast(lat, lng, radiusKm);
 const trend =
 reports.length > reports_previous_24h.length * 1.2 ? 'rising'
:
 reports.length < reports_previous_24h.length * 0.8 ?
'falling' : 'stable';

 return {
 reports,
 summary: {
 totalThreats: reports.length,
 threatLevel: localThreatLevel,
 topScamTypes,
 trend,
 last24hIncrease: reports.length -
reports_previous_24h.length
 }
 };
}

```

## 4. Subscription Billing Logic

```

// Stripe Webhook Handler
async function handleStripeWebhook(event: Stripe.Event) {
 switch (event.type) {
 case 'checkout.session.completed': {
 const session = event.data.object as
Stripe.Checkout.Session;

 // Create subscription record
 await db.subscriptions.insert({
 user_id: session.client_reference_id,
 plan_type: session.metadata.plan_type,

```

```

 amount: session.amount_total / 100,
 status: 'active',
 stripe_subscription_id: session.subscription,
 stripe_customer_id: session.customer,
 current_period_start: new Date(),
 current_period_end: new Date(Date.now() +
getPeriodDuration(session.metadata.plan_type))
 });

 // Update user role
 await db.users.update(session.client_reference_id, {
 role: 'analyst',
 subscription_plan: session.metadata.plan_type,
 subscription_status: 'active'
 });

 // Send welcome email
 await sendEmail({
 to: session.customer_email,
 template: 'analyst_welcome',
 data: { plan: session.metadata.plan_type }
 });

 break;
}

case 'invoice.payment_succeeded': {
 const invoice = event.data.object as Stripe.Invoice;

 // Record payment
 await db.payments.insert({
 subscription_id: invoice.subscription,
 user_id: await
getUseridFromStripeCustomer(invoice.customer),
 amount: invoice.amount_paid / 100,
 payment_status: 'completed',
 stripe_payment_intent_id: invoice.payment_intent,
 invoice_url: invoice.hosted_invoice_url
 });

 break;
}

case 'invoice.payment_failed': {
 const invoice = event.data.object as Stripe.Invoice;

 // Update subscription status
 await db.subscriptions.update(
 { stripe_subscription_id: invoice.subscription },
 { status: 'past_due' }
);
}

```

```

 // Send payment failed email
 await sendEmail({
 to: invoice.customer_email,
 template: 'payment_failed',
 data: { amount: invoice.amount_due / 100 }
 });

 break;
 }

 case 'customer.subscription.deleted': {
 const subscription = event.data.object as
Stripe.Subscription;

 // Mark subscription as cancelled
 await db.subscriptions.update(
 { stripe_subscription_id: subscription.id },
 {
 status: 'cancelled',
 cancelled_at: new Date()
 }
);

 // Downgrade user role
 await db.users.update(
 { stripe_customer_id: subscription.customer },
 {
 role: 'public',
 subscription_plan: 'free',
 subscription_status: null
 }
);

 break;
 }
}

function getPeriodDuration(plan: 'monthly' | 'annual'): number {
 return plan === 'monthly'
 ? 30 * 24 * 3600 * 1000 // 30 days
 : 365 * 24 * 3600 * 1000; // 365 days
}

```

## 5. Email Service Integration

```

// Newsletter Sending
async function sendNewsletter(newsletterId: string) {
 const newsletter = await db.newsletters.findById(newsletterId);
 const subscribers = await db.newsletter_subscribers.find({
 status: 'active'
 });
}

```

```

});

// Update status
await db.newsletters.update(newsletterId, {
 status: 'sending',
 recipient_count: subscribers.length
});

// Send in batches (1000 at a time)
for (let i = 0; i < subscribers.length; i += 1000) {
 const batch = subscribers.slice(i, i + 1000);

 await sendGridClient.send({
 personalizations: batch.map(sub => ({
 to: [{ email: sub.email }],
 custom_args: {
 newsletter_id: newsletterId,
 subscriber_id: sub.id
 }
 })),
 from: { email: 'newsletter@crowdshield.com', name:
'CrowdShield' },
 subject: newsletter.subject,
 content: [
 { type: 'text/plain', value: newsletter.content },
 { type: 'text/html', value: newsletter.html_content ||
newsletter.content }
],
 tracking_settings: {
 click_tracking: { enable: true },
 open_tracking: { enable: true }
 }
 });

 // Update sent count
 await db.newsletters.update(newsletterId, {
 sent_count: i + batch.length
 });

 // Final update
 await db.newsletters.update(newsletterId, {
 status: 'sent',
 sent_at: new Date()
 });
}

// SendGrid Webhook Handler (track opens/clicks)
async function handleSendGridWebhook(events: any[]) {
 for (const event of events) {
 const { newsletter_id, subscriber_id } = event.custom_args ||
{};

```



```

 if (!newsletter_id) continue;

 switch (event.event) {
 case 'delivered':
 await db.newsletters.increment(newsletter_id,
'delivered_count');
 break;

 case 'open':
 await db.newsletters.increment(newsletter_id,
'opened_count');
 if (subscriber_id) {
 await
db.newsletter_subscribers.increment(subscriber_id,
'total_opened');
 }
 break;

 case 'click':
 await db.newsletters.increment(newsletter_id,
'clicked_count');
 if (subscriber_id) {
 await
db.newsletter_subscribers.increment(subscriber_id,
'total_clicked');
 }
 break;

 case 'unsubscribe':
 await db.newsletters.increment(newsletter_id,
'unsubscribed_count');
 if (subscriber_id) {
 await db.newsletter_subscribers.update(subscriber_id, {
 status: 'unsubscribed',
 unsubscribed_at: new Date()
 });
 }
 break;
 }
 }

 // Recalculate rates
 const newsletter = await
db.newsletters.findById(newsletter_id);
 await db.newsletters.update(newsletter_id, {
 delivery_rate: (newsletter.delivered_count /
newsletter.sent_count) * 100,
 open_rate: (newsletter.opened_count /
newsletter.delivered_count) * 100,
 click_rate: (newsletter.clicked_count /
newsletter.opened_count) * 100,

```

```

 unsubscribe_rate: (newsletter.unsubscribed_count /
newsletter.delivered_count) * 100
 });
}

```

## FRONTEND COMPONENTS

### Component Structure

```

src/
├── app/
│ ├── App.tsx # Main app with routing logic
│ └── components/
│ ├── SeverityBadge.tsx # Reusable severity indicator
│ ├── ScamTypeTag.tsx # Scam type pill
│ ├── AiScoreRing.tsx # Circular progress ring
│ ├── VoteButtons.tsx # Vote interface
│ ├── MetricCard.tsx # Dashboard stat cards
│ ├── ReportCard.tsx # Report display card
│ ├── LivePulseDot.tsx # Animated live indicator
│ ├── TopNav.tsx # Top navigation bar
│ ├── Sidebar.tsx # Analyst sidebar nav
│ ├── AdminSidebar.tsx # Admin sidebar nav
│ ├── PublicNav.tsx # Public navigation
│ ├── NewsletterSignup.tsx # Newsletter form
│ ├── SimulationModal.tsx # Simulation config modal
│ ├── SimulationBanner.tsx # Active simulation banner
│ ├── PaymentModal.tsx # Payment form
│ └── LoginModal.tsx # Login dialog
│ └── screens/
│ ├── PublicLanding.tsx # Home page
│ ├── PublicFeed.tsx # Public live feed
│ ├── PublicSubmitReport.tsx # Public report form
│ ├── MyLocationThreats.tsx # Location-based view
│ ├── Pricing.tsx # Pricing page
│ ├── Dashboard.tsx # Analyst dashboard
│ ├── LiveFeed.tsx # Analyst live feed
│ ├── GeoMap.tsx # Geo intelligence map
│ ├── Campaigns.tsx # Campaign tracking
│ ├── Analytics.tsx # Analytics charts
│ ├── Alerts.tsx # Alert center
│ └── SubmitReport.tsx # Analyst report form
│ └── admin/
│ ├── AdminDashboard.tsx # Admin overview
│ ├── AdminUsers.tsx # User management
│ ├── AdminReports.tsx # Report moderation
│ ├── AdminSubscriptions.tsx # Subscription management
│ ├── AdminMessages.tsx # Support messages
│ ├── AdminAnnouncements.tsx # Announcements
│ └── AdminNewsletter.tsx # Newsletter management
└── data/

```

```
└─ mockData.ts # Mock data for demo
└─ styles/
 └─ theme.css # Color palette & variables
 └─ fonts.css # Typography imports
```



## DEPLOYMENT GUIDE

### 1. Environment Variables

```
.env.local

Database
CONVEX_DEPLOYMENT_URL=https://your-deployment.convex.cloud
CONVEX_DEPLOY_KEY=your_deploy_key

Azure AI
AZURE_OPENAI_API_KEY=your_azure_key
AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT_NAME=gpt-4-turbo

Stripe
STRIPE_SECRET_KEY=sk_live_...
STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_MONTHLY_PRICE_ID=price_...
STRIPE_ANNUAL_PRICE_ID=price_...

Email Service
SENDGRID_API_KEY=SG....
SENDGRID_FROM_EMAIL=noreply@crowdshield.com
OR
AWS_SES_ACCESS_KEY=...
AWS_SES_SECRET_KEY=...
AWS_SES_REGION=us-east-1

Storage
AWS_S3_BUCKET=crowdshield-uploads
AWS_S3_ACCESS_KEY=...
AWS_S3_SECRET_KEY=...
AWS_S3_REGION=us-east-1

App
NEXT_PUBLIC_APP_URL=https://crowdshield.com
SESSION_SECRET=your_random_secret_key
JWT_SECRET=your_jwt_secret

Monitoring
DATADOG_API_KEY=...
DATADOG_APP_KEY=...
```

## 2. Deployment Steps

# 1. Install dependencies

```
npm install # or pnpm install
```

# 2. Set up Convex

```
npx convex dev # for development
```

```
npx convex deploy # for production
```

# 3. Run database migrations

```
npm run db:migrate
```

# 4. Seed initial data

```
npm run db:seed
```

# 5. Build frontend

```
npm run build
```

# 6. Deploy to Vercel

```
vercel --prod
```

# 7. Set up Stripe webhooks

```
stripe listen --forward-to https://crowdshield.com/api/webhooks/stripe
```

# 8. Set up SendGrid webhooks

```
Point to: https://crowdshield.com/api/webhooks/sendgrid
```

# 9. Configure CDN

```
Point Cloudflare to Vercel deployment
```

# 10. Set up monitoring

```
Configure DataDog dashboard
```

## 3. CI/CD Pipeline (GitHub Actions)

```
.github/workflows/deploy.yml
```

```
name: Deploy to Production
```

```
on:
```

```
 push:
```

```
 branches: [main]
```

```
jobs:
```

```
 deploy:
```

```
 runs-on: ubuntu-latest
```

```
 steps:
```

```
 - uses: actions/checkout@v3
```

```
- name: Setup Node.js
 uses: actions/setup-node@v3
 with:
 node-version: '18'
```

```
- name: Install dependencies
 run: npm ci
```

```
- name: Run tests
 run: npm test
```

```
- name: Build
 run: npm run build
 env:
 CONVEX_DEPLOYMENT_URL: $
 {{ secrets.CONVEX_DEPLOYMENT_URL }}
```

```
- name: Deploy to Vercel
 uses: amondnet/vercel-action@v25
 with:
 vercel-token: ${{ secrets.VERCEL_TOKEN }}
 vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
 vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
 vercel-args: '--prod'
```

```
- name: Deploy Convex
 run: npx convex deploy
 env:
 CONVEX_DEPLOY_KEY: ${{ secrets.CONVEX_DEPLOY_KEY }}
```



## DEVELOPMENT CHECKLIST FOR CURSOR

When building this in Cursor, follow this order:

### Phase 1: Foundation (Week 1)

- Set up React + TypeScript + Tailwind project
- Configure Convex database
- Create theme system (colors, fonts)
- Build component library (badges, cards, buttons)
- Set up routing

### Phase 2: Public Features (Week 2)

- Public landing page
- Report submission form
- AI integration (Azure OpenAI)
- Live feed with voting
- Location-based threats
- Newsletter signup

### **Phase 3: Subscription System (Week 3)**

- Pricing page
- Stripe integration
- Payment modal
- Subscription management
- Role-based access control

### **Phase 4: Analyst Features (Week 4-5)**

- Analyst dashboard
- Geo intelligence map
- Campaign tracking
- Analytics charts
- Alert system
- Simulation mode

### **Phase 5: Admin Panel (Week 6-7)**

- Admin dashboard
- User management
- Report moderation
- Subscription management
- Messages & support
- Announcements
- Newsletter management

### **Phase 6: Polish & Deploy (Week 8)**

- Email service integration
- Webhook handlers
- Testing
- Performance optimization
- Security audit
- Production deployment

This specification provides everything you need to build CrowdShield in Cursor. Start with Phase 1 and work your way through systematically! 🚀